

# React Compiler: Incremental Adoption

**Incremental adoption** ka simple meaning hai: React Compiler ko poore project par ek saath enable karne ke bajay, **gradually chhote-chhote parts par apply karna**.

Is approach se aap pehle compiler ko limited components ya directories par test kar sakte ho. Jab confidence build ho jaye aur sab correctly work kare, tab compiler ka coverage expand kar sakte ho.

---

## React Compiler ko gradually adopt kyun karein?

React Compiler automatically poore codebase ko optimize karne ke liye design kiya gaya hai, lekin ise ek hi baar mein har jagah enable karna compulsory nahi hai.

Incremental adoption ke kuch important benefits hain:

### 1. Chhote level par testing

Pehle app ke kisi ek small section par compiler enable karke check kar sakte ho:

- App ka behaviour same reh raha hai ya nahi
- Compiled code properly work kar raha hai ya nahi
- Performance mein improvement aa rahi hai ya nahi
- Aapke project se related koi special edge case toh nahi aa raha

Production applications mein stability important hoti hai, isliye small rollout comparatively manageable hota hai.

### 2. Rules of React violations gradually fix kar sakte ho

Compiler aapke code mein **Rules of React violations** identify kar sakta hai.

Agar poore codebase par ek saath compiler enable karoge, toh ek hi baar mein bahut saare violations fix karne pad sakte hain. Incremental rollout mein aap directory ya component ke hisaab se violations systematically fix kar sakte ho.

### 3. Migration manageable rehti hai

Ek saath poora codebase migrate karne ke bajay, migration ko small parts mein divide kiya ja sakta hai. Isse bugs introduce hone ka risk reduce hota hai.

### 4. A/B testing kar sakte ho

Aap compiler-enabled aur original code ko different users ke liye run karke compare kar sakte ho. Isse compiler optimizations ka real-world impact measure karna easier hota hai.

Ye data team ko full adoption ka decision lene mein bhi help kar sakta hai.

---

## Incremental Adoption ke 3 main approaches

React Compiler ko gradually adopt karne ke teen primary approaches hain:

1. **Babel overrides**

Compiler ko selected directories par apply karna.

2. **"use memo" ke saath opt-in**

Sirf un components aur hooks ko compile karna jo explicitly compiler ko enable karte hain.

3. **Runtime gating**

Feature flags ke through runtime par optimized aur original code ke beech control karna.

Teeno approaches ka goal same hai: compiler ko full rollout se pehle limited code par safely test karna.

---

### 1. Babel Overrides ke saath Directory-Based Adoption

Babel ka overrides option different files ya directories par different plugins apply karne deta hai.

Ye approach tab useful hai jab aap directory-by-directory compiler adopt karna chahte ho.

#### Basic configuration

```
1 // babel.config.js
2 module.exports = {
3   plugins: [
4     // Global plugins jo sabhi files par apply honge
5   ],
6   overrides: [
7     {
8       test: './src/modern/**/*.{js,jsx,ts,tsx}',
9       plugins: [
10         'babel-plugin-react-compiler'
11       ]
12     }
13   ]
14 };
```

Is configuration mein React Compiler sirf:

```
1 src/modern/
```

directory ke andar maujood in file types par apply hoga:

- .js
- .jsx
- .ts
- .tsx

Baaki project compiler se affect nahi hoga.

### test ka role

```
1 test: './src/modern/**/*.{js,jsx,ts,tsx}'
```

test decide karta hai ki override kin files par apply hona chahiye.

### plugins ka role

```
1 plugins: [  
2   'babel-plugin-react-compiler'  
3 ]
```

Is directory ke matching files ke liye React Compiler ka Babel plugin enable kiya ja raha hai.

---

## Coverage expand karna

Jab initial directory par compiler properly work karne lage, toh additional directories include ki ja sakti hain:

```
1 // babel.config.js  
2 module.exports = {  
3   plugins: [  
4     // Global plugins  
5   ],  
6   overrides: [  
7     {  
8       test: [  
9         './src/modern/**/*.{js,jsx,ts,tsx}',  
10        './src/features/**/*.{js,jsx,ts,tsx}'  
11      ],  
12      plugins: [  
13        'babel-plugin-react-compiler'  
14      ]  
15    },  
16    {
```

```
17     test: './src/legacy/**/*.{js,jsx,ts,tsx}',
18     plugins: [
19         // Legacy code ke liye different plugins
20     ]
21 }
22 ]
23 };
```

Ab compiler do directories ke code par apply hoga:

```
1  src/modern/
2  src/features/
```

Lekin:

```
1  src/legacy/
```

directory ko separately handle kiya ja raha hai. Legacy code ke liye different Babel plugins rakhe ja sakte hain.

Is tarah modern ya stable code par compiler enable karke old/legacy code ko temporarily exclude kiya ja sakta hai.

---

## Different directories ke liye compiler options

Har override ke andar compiler ko alag options ke saath configure kiya ja sakta hai:

```
1  // babel.config.js
2  module.exports = {
3    plugins: [],
4    overrides: [
5      {
6        test: './src/experimental/**/*.{js,jsx,ts,tsx}',
7        plugins: [
8          ['babel-plugin-react-compiler', {
9            // Experimental directory ke options
10          }]
11      ]
12    },
13    {
14      test: './src/production/**/*.{js,jsx,ts,tsx}',
15      plugins: [
16        ['babel-plugin-react-compiler', {
17          // Production directory ke options
18        }]
19      ]
20    }
21  ]
22 }
```

```
19     ]
20   }
21 ]
22 };
```

Yahan:

- src/experimental/ ke liye ek compiler configuration ho sakti hai.
- src/production/ ke liye doosri compiler configuration ho sakti hai.

Isse project ke different areas ke liye separate compiler behaviour control kiya ja sakta hai.

---

## 2. "use memo" ke saath Opt-in Mode

Agar directory-level control se bhi zyada fine-grained control chahiye, toh:

```
1  compilationMode: 'annotation'
```

use kiya ja sakta hai.

Is mode mein compiler automatically har component ko compile nahi karega. Sirf wahi component ya custom hook compile hoga jiske andar explicitly:

```
1  "use memo";
```

likha hoga.

---

## Annotation mode configuration

```
1  // babel.config.js
2  module.exports = {
3    plugins: [
4      ['babel-plugin-react-compiler', {
5        compilationMode: 'annotation',
6      }],
7    ],
8  };
```

Yahan compilationMode ko 'annotation' set kiya gaya hai.

Iska effect ye hai ki compiler ko har function ke andar annotation/directive ke through batana padega ki use compile karna hai.

---

## Component mein "use memo" use karna

```
1 function TodoList({ todos }) {
2   "use memo"; // Is component ko compilation mein opt-in karo
3
4   const sortedTodos = todos.slice().sort();
5
6   return (
7     <ul>
8       {sortedTodos.map(todo => (
9         <TodoItem key={todo.id} todo={todo} />
10      ))}
11     </ul>
12   );
13 }
```

Function ke beginning mein:

```
1 "use memo";
```

likhne ki wajah se TodoList component compiler ke liye opt-in ho gaya.

Compiler is component ko optimize kar sakta hai, jabki bina directive wale components compile nahi honge.

---

## Custom hook mein "use memo"

```
1 function useSortedData(data) {
2   "use memo"; // Is custom hook ko compilation mein opt-in karo
3
4   return data.slice().sort();
5 }
```

"use memo" sirf React components ke liye nahi hai. Isse custom hooks ke beginning mein bhi add kiya ja sakta hai.

Is example mein useSortedData hook explicitly compilation ke liye opt-in kar raha hai.

---

## Annotation mode mein kya yaad rakhna hoga?

compilationMode: 'annotation' use karte waqt:

- Har optimize kiye jaane wale component mein "use memo" add karna hoga.
- Har optimize kiye jaane wale custom hook mein "use memo" add karna hoga.

- Future mein banne wale new components aur hooks mein bhi directive manually add karna yaad rakhna hoga.

### Is approach ka main advantage

Aap individual component aur hook level par decide kar sakte ho ki compiler kahan run karega.

### Iska trade-off

Manual work badh jata hai, kyunki har eligible function mein "use memo" explicitly add karna padta hai.

---

## "use no memo" ka purpose

Agar compiler adopt karte waqt koi specific component problem create kar raha hai, toh us component ko temporarily compilation se exclude karne ke liye:

```
1 "use no memo";
```

use kiya ja sakta hai.

Conceptually:

```
1 function ProblematicComponent() {  
2   "use no memo";  
3  
4   // Component implementation  
5 }
```

Ye useful hai jab:

- Compiler mostly project par correctly work kar raha ho
- Sirf kuch components mein issue aa raha ho
- Aap poora compiler disable nahi karna chahte
- Problematic component ko temporarily exclude karke baaki adoption continue rakhna chahte ho

Provided content mein "use no memo" ka complete implementation example nahi diya gaya, lekin troubleshooting option ke roop mein ise problematic components ko temporarily exclude karne ke liye mention kiya gaya hai.

---

### 3. Runtime Feature Flags with Gating

gating option compiler adoption ko runtime feature flag ke through control karne deta hai.

Ye specially useful hai:

- A/B testing ke liye
  - Gradual rollout ke liye
  - Different user segments par compiler test karne ke liye
- 

#### Gating kaise kaam karta hai?

Compiler optimized code ko ek runtime check ke saath wrap karta hai.

- Agar gating function true return kare, toh optimized version run hota hai.
- Agar gating function false return kare, toh original version run hota hai.

Isse application ko rebuild kiye bina feature flag ke basis par compiled behaviour control kiya ja sakta hai.

---

#### Gating configuration

```
1 // babel.config.js
2 module.exports = {
3   plugins: [
4     ['babel-plugin-react-compiler', {
5       gating: {
6         source: 'ReactCompilerFeatureFlags',
7         importSpecifierName: 'isCompilerEnabled',
8       },
9     ]],
10  ],
11 };
```

##### source

```
1 source: 'ReactCompilerFeatureFlags'
```

Ye us module ka naam hai jahan gating function defined hai.

##### importSpecifierName

```
1 importSpecifierName: 'isCompilerEnabled'
```

Ye us exported function ka naam hai jise compiler runtime check ke liye use karega.

---

## Feature flag function implement karna

```
1 // ReactCompilerFeatureFlags.js
2 export function isCompilerEnabled() {
3   // Apna feature flag system use karo
4   return getFeatureFlag('react-compiler-enabled');
5 }
```

Yahan isCompilerEnabled() function feature flag system se compiler ki current state read karta hai.

Agar:

```
1 getFeatureFlag('react-compiler-enabled')
```

true return karega, toh optimized version use hoga.

Agar false return karega, toh original code use hoga.

---

## Kaunsa approach kab use karenin?

### Babel overrides

Jab aapka codebase clearly directories mein divided ho aur aap directory-by-directory rollout karna chahte ho.

Example:

```
1 src/modern/
2 src/features/
3 src/legacy/
```

### "use memo" annotation mode

Jab aapko individual component aur custom hook level par precise control chahiye.

Ye tab useful hai jab aap sirf kuch selected components par compiler test karna chahte ho.

### Runtime gating

Jab optimized code ko feature flags ke through runtime par control karna ho.

Ye A/B testing aur user-segment-based rollout ke liye useful hai.

In approaches ko strictly isolated alternatives samajhna zaroori nahi hai. Provided content ke according, sabka common purpose full rollout se pehle application ke selected parts par compiler test karna hai.

---

## Adoption ke waqt issues aayein toh kya karein?

Provided troubleshooting steps ke according:

### 1. Problematic component exclude karo

Temporary exclusion ke liye:

```
1 "use no memo";
```

use karo.

### 2. Debugging guide check karo

Common compiler issues identify aur troubleshoot karne ke liye debugging guide refer karo.

### 3. Rules of React violations fix karo

ESLint plugin compiler-related Rules of React violations identify kar sakta hai. Un violations ko fix karna adoption process ka part ho sakta hai.

### 4. Annotation mode consider karo

Agar normal rollout mein issues aa rahe hain, toh:

```
1 compilationMode: 'annotation'
```

use karke compiler ko selected components aur hooks tak limit kar sakte ho.

---

## Practical Adoption Flow

Provided approaches ko ek practical sequence mein samjho:

### Step 1: Small area select karo

Ek stable ya controlled directory choose karo:

```
1 src/modern/
```

### Step 2: Babel override se compiler enable karo

Sirf selected directory par babel-plugin-react-compiler apply karo.

### Step 3: Behaviour verify karo

Check karo ki compiled code ke saath application correctly behave kar rahi hai.

### Step 4: Performance impact observe karo

Compiler ki optimizations se performance mein kya impact aa raha hai, use measure karo.

### Step 5: Violations address karo

Compiler ya ESLint plugin se identify hone wale Rules of React violations fix karo.

### Step 6: Problematic components exclude karo

Temporary problems ke liye "use no memo" use karo.

### Step 7: Coverage expand karo

Confidence build hone ke baad additional directories ko Babel override mein include karo.

### Step 8: Runtime gating consider karo

Agar A/B testing ya user-based rollout chahiye, toh feature-flag-based gating configure karo.

---

## Short Summary

React Compiler ko ek saath poore project mein enable karna compulsory nahi hai. Aap ise gradually adopt kar sakte ho:

- **Babel overrides** se selected directories compile karo.
- `compilationMode: 'annotation'` aur "use memo" se selected components/hooks ko opt-in karao.
- "use no memo" se problematic components temporarily exclude karo.
- **Runtime gating** se feature flags ke basis par optimized ya original code run karao.
- Pehle small rollout karo, behaviour aur performance verify karo, violations fix karo aur phir compiler coverage expand karo.

Overall, incremental adoption ka purpose React Compiler migration ko **controlled, testable aur manageable** banana hai.

## External links

Provided content mein Mozilla/MDN ke alawa koi external website link included nahi hai. Saare mentioned documentation links internal relative links hain.